Extracting, Storing, and Ingesting Data

Data Engineering (CMP020X304S) Coursework 1 Report

Evangeline Anderson (AND22529596
Micail Bakhshzaad (BAK22529047
Catherine Binyamin (BIN22555039),
Mariam Hersi (HER22555014),
Zeeshan Kazmi (KAZ22527866)

***Abstract*— This project implements a data engineering pipeline that extracts book data from the Packt website using web scraping, processes it, and stores it in both MySQL and MongoDB databases. Apache NiFi is used to automate data flow and format conversion, with a comparison of query performance between relational and non-relational databases.**

I. Introduction

The website Packt sells nearly 200 paperback and eBooks for 'data engineering' [1], each with key information about the book such as star ratings and detailed descriptions. An overview of the implementation method and its explanation will be found below.

Task 1 began with selecting our website and downloading the webpage using BeautifulSoup. Once parsed and extracted, a data frame was built using Pandas. The final data extracted was saved as CSV data, ready for the Task 2 MySQL database setup.

Task 2 involved setting up a MySQL database to effectively store and handle the data that has been scraped. Draw.io was utilised to develop a database schema for a table Engineering_Books. The CSV file from Task 1 was imported into MySQL and stored as a table called Engineering_Books in our database Packt_db. Our final task was to execute an SQL query to extract only three columns from the table and sort the table based on any column.

Task 3 required us to use Apache Nifi. The data from the table was extracted and converted to JSON format using multiple processors and stored into a file called Data Engineering on our local disk.

Task 4 comprised us converting CSV to JSON, then storing the JSON data in a MongoDB database as a collection. We then queried the MongoDB database and compared query execution time between MongoDB and MySQL.

II. Task 1 – Web Scraping

Through scraping, information was ascertained: ISBN, title, author(s), publication date, star rating, number of ratings, price, book format (eBook or paperback) and book URL. Information on 59 books was subsequently stored in a CSV file for further processing.

A. Web Scraping

In our initial program, we sequentially retrieved data for each book using a for loop to iterate through each book URL.

Processing took 1 minute.

```
[8] # Empty list to store book dictionaries
    book_info_list = []

    # Iterate through book urls and gather information on both
    for url in book_urls:
        # Get the book information for the current url
        book_info = get_book_information(url)
        # Append book data to the list
        book_info_list.append(book_info)

    # Convert list of dictionaries to pandas DataFrame
    df = pd.DataFrame(book_info_list)

    # Convert to csv file
    df.to_csv("engineering_books_from_Packt.csv", index=False)
```

B. Multithreading

Threads enable parallel execution, allowing a program to run tasks simultaneously. Threads improve the performance of applications that involve I/O operations, such as making web requests or reading data from files. [2]. The concurrent.futures library was used to provide access to the ThreadPoolExecutor() function [3].

By using multithreading to send 10 requests at a time, processing the 59 books took 22 seconds.

```
[8] # Empty list to store book dictionaries
    book_info_list = []

    # Iterate through book urls and gather information on both
    for url in book_urls:
        # Get the book information for the current url
        book_info = get_book_information(url)
        # Append book data to the list
        book_info_list.append(book_info)

    # Convert list of dictionaries to pandas DataFrame
    df = pd.DataFrame(book_info_list)

    # Convert to csv file
    df.to_csv("engineering_books_from_Packt.csv", index=False)
```

C. Comparison

The key difference between multiprocessing and multithreading is that multiprocessing creates separate processes, each with its own memory space, running in parallel. This takes advantage of multi-core processors. In contrast, multithreading runs multiple threads within a single process, sharing the same memory space [5].
Multithreading took 22 seconds to execute the code, which was significantly faster than when the web scraping was performed sequentially (1 minute).
On the other hand, multiprocessing took 18 seconds which was faster than when using multithreading. Overall, if more books were added to our extraction, the difference in speed might've been more noticeable.

III. Task 2 – Transfer to MySQL

A. Schema

The data retrieved from the web scraping fit into our database schema as shown below.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

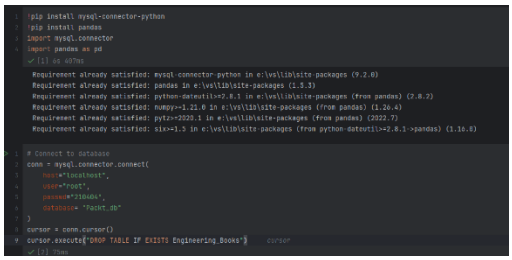
engineering_books		
PK	isbn	varchar(20) NOT NULL
	book_title	text NOT NULL
	author_names	varchar(255) NOT NULL
	publication_date	date NOT NULL
	star_rating	float
	num_of_ratings	int
	price	float NOT NULL
	format	varchar(10) NOT NULL
	url	varchar(255) NOT NULL

We set the primary key to be the books' ISBN, a product identifier used in relation to books [6]. This served as a unique identifier for each book.

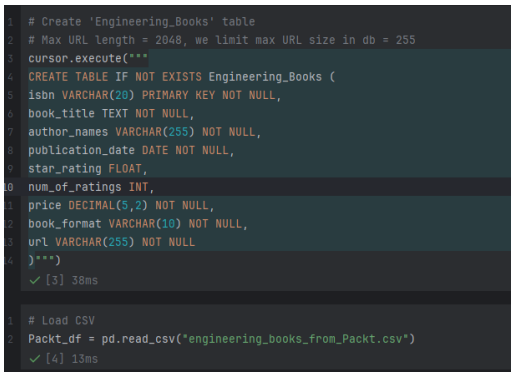
Each book's name, author, publication date, rating, number of ratings, price, format (paperback or audiobook), and URL link are all represented as fields in the schema. Star rating and number of ratings remain nullable to account for books which have not received any reviews yet.

B. *Python Code*

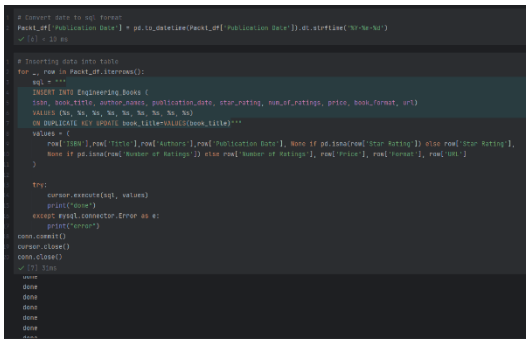
In the Jupyter notebook, a MySQL connector was imported, following a W3Schools guide [7], and a connection was made to localhost:3306.



The table 'Engineering_Books' was created. To fill the table with the web scraped data from Task 1, we imported the CSV file using the Pandas library.



The formatted date would have been a simple string field in MySQL, so we had to convert the date to work in MySQL.

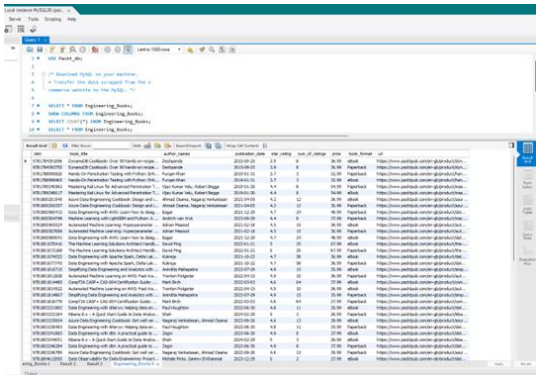


C. *Querying the Database*

To see the table in Workbench, we use the queries:

USE Packt_db;

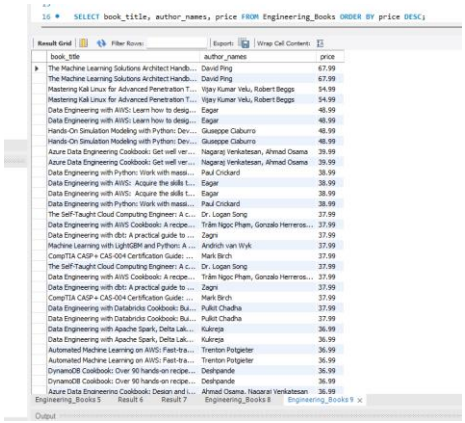
SELECT * FROM Engineering_Books;



To extract only three columns from the table and sort the table based on any column, we used the query:

SELECT book_title, author_names, price FROM Engineering_Books ORDER BY price DESC;

By retrieving the book title, author names, and price, this query sorted the books in descending order of price, making it easier to find the most costly books first.



IV. Task 3 – Extraction using Apache NiFi

Using the ExecuteSQL, ConvertRecord, and PutFile processors, along with a DBCP connection pool, the contents of the MySQL database were extracted and converted to a clean and readable, structured JSON file format, stored in a Data Engineering folder on the local disk. The DBCP

connection pool was provided with connecter/J, a driver officially provided by MySQL to enable communication with databases created using it [8].

V. Task 4 – MongoDB Integration

A. Converting to JSON and storing in MongoDB

We first converted the CSV data into JSON format in the jupyter notebook. This created a JSON array of each book. We then used MongoDB to create a database and a collection using the JSON data.

B. Comparison of MongoDB and MySQL

The following table shows each of the queries which were run in MySQL and MongoDB, with a comparison of the execution times of each.

Query description	MySQL query	Execution time (milliseconds)	MongoDB query	Execution time (milliseconds)
Show all rows	SELECT * FROM engineering_books;	0.34510	db["books_collection"].find().explain("executionStats")	1
Search for specific title	SELECT * FROM engineering_books WHERE book_title = "Data Engineering with Databricks Cookbook: Build effective data and AI solutions using Apache Spark, Databricks, and Delta Lake";	0.46410	db["books_collection"].find({**"Title": "Data Engineering with Databricks Cookbook: Build effective data and AI solutions using Apache Spark, Databricks, and Delta Lake"**}).explain("executionStats")	2
Show rows with a star rating > 3	SELECT * FROM engineering_books WHERE star_rating > 3;	0.45030	db["books_collection"].find({**"Star Rating": {$gt: 3}**}).explain("executionStats")	1
Show rows with "AWS" in title	SELECT * FROM engineering_books WHERE book_title LIKE '%AWS%';	0.33920	db["books_collection"].find({**"Title": {"$regex": "AWS", "$options": "i"}**}).explain("executionStats")	0
Show rows where star rating is NULL	SELECT * FROM engineering_books WHERE star_rating IS NULL;	0.31940	db["books_collection"].find({**"Star Rating": null** }).explain("executionStats")	0

The findings indicate that MySQL tends to perform queries faster than MongoDB, however it should be noted that MongoDB rounds the execution time values which could mean that the real query execution times are faster than what we're seeing.

C. Applying indexing

An index in a database is a data structure that improves the speed of queries by allowing the database to find and retrieve data more efficiently.

Indexes help MongoDB run queries more efficiently. Without them, MongoDB has to scan every document in a collection to find results. When an index is available, MongoDB uses it to narrow down the search, reducing the number of documents it needs to check [9]. When searching through the collections without using indexing, MongoDB uses "COLLSCAN" which means it is scanning the whole collection. When viewing the execution stats, we can see that "executionTimeMillis" is slower than when indexing is used.

Index description	MySQL index	Execution Time	MongoDB index	Execution Time
Title indexing on ascending order	CREATE FULLTEXT INDEX idx_book_title ON engineering_books(book_title);	0.2437	db["books_collection"].createIndex({ "Title": 1 })	0
Unique indexing on ISBN	CREATE UNIQUE INDEX idx_isbn ON engineering_books (isbn ASC);	0.72250	db["books_collection"].createIndex({ "ISBN": 1 }, { unique: true })	0
Text indexing on book title and authors	CREATE FULLTEXT INDEX idx_text_search ON engineering_books (book_title);	0.2347	db["books_collection"].createIndex({ "Title": "text", "Authors": "text" })	0
Sparse indexing on star rating	ALTER TABLE engineering_books ADD COLUMN star_rating_index INT GENERATED ALWAYS AS (CASE WHEN `star_rating` IS NOT NULL THEN `star_rating` ELSE NULL END) STORED; CREATE INDEX idx_star_rating ON engineering_books(star_rating_index);	0.4289	We shouldn't use this as this index is not good for finding documents where star is missing because it only indexes documents where "Star Rating" exists (i.e., does not index documents where "Star Rating" is null or missing).	0
Compound indexing on book title and author names	CREATE FULLTEXT INDEX idx_book_title_author ON engineering_books(book_title, author_names);	0.3667	db["books_collection"].createIndex({ "Title": 1, "Authors": 1 });	0

Acknowledgment *(Heading 5)*

I would like to express my sincere gratitude to my professor for their guidance, support, and valuable feedback throughout this project. Their insights and encouragement were instrumental in helping me develop a deeper understanding of data engineering concepts and successfully complete this work.

References

[1] Packt, "Packt Search Results for 'data engineering,'" 2025. Available: https://www.packtpub.com/en-gb/search?q=data%20engineering

[2] D. Ocean, "How To Use ThreadPoolExecutor in Python 3," Jun. 2020. Available: https://www.digitalocean.com/community/tutorials/how-to-use-threadpoolexecutor-in-python-3

[3] P. S. Foundation, "concurrent.futures - Launching parallel tasks - ThreadPoolExecutor," Mar. 2025. Available: https://docs.python.org/3/library/concurrent.futures.html#concurrent.futures.ThreadPoolExecutor

[4] P. S. Foundation, "concurrent.futures - Launching parallel tasks - ProcessPoolExecutor," Mar. 2025. Available: https://docs.python.org/3/library/concurrent.futures.html#concurrent.futures.ProcessPoolExecutor

[5] N. Sharma, "Multiprocessing vs Multithreading in Python," Sep. 2023. Available: https://www.nimbleappgenie.com/blogs/multiprocessing-vs-multithreading

[6] I. I. Agency, "What is an ISBN?" Available: https://www.isbn-international.org/content/what-isbn/10

[7] W3Schools, "Install MySQL Driver." Available: https://www.w3schools.com/python/python_mysql_getstarted.asp

[8] Oracle, "MySQL Community Downloads - Connector/J 9.2.0," 2025. Available: https://dev.mysql.com/downloads/connector/j

[9] M. Inc, "MongoDB manual - Indexes." Available: https://www.mongodb.com/docs/manual/indexes/?utm_source=compass&utm_medium=product#single-fie